NP-Completeness

Algorithms Final Report

Angel Coronel
Computer Science / Algorithms
CUNY

***Abstract*—This report covers Chapter 34 of Introduction to Algorithms (CLRS) [1], NP-completeness. The goal of is to show how computational problems are classified by tractability and how hardness is proven with polynomial time reductions. The complexity classes P and NP, NP-hardness and NP-completeness are covered. This report includes a NP-complete proof using a known NP-complete problem. Finally, I go over the implications of NP-completeness for algorithm design.**

***Index Terms*—NP-completeness, reductions, P, NP**

I. Introduction

Something we see time and time again in algorithm design is the gap between finding a solution and then checking a proposed solution. There are a lot of problems in graphs and scheduling where a proposed solution can be checked very quickly if its correct or not but no polynomial time algorithm is known for coming up with that solution in the worst case. In Chapter 34 of Introduction to Algorithms (CLRS), they show this with the Hamiltonian Cycle problem (while exhaustive search is super-polynomial, a given Hamiltonian cycle can be verified in polynomial time by checking vertex uniqueness and edge existence).

NP-completeness is a formal way to classify the kind of problems where it's easy to verify but hard to find. There is also a rigorous method using polynomial time reductions to justify why efficient algorithms might not exists. NP-completeness shows that a polynomial time algorithm for one problem would imply polynomial time algorithms for an entire class of problems.

CLRS treats polynomial time as the benchmark for efficiency. The main tool they use if a polynomial time many to one reduction $L_1 \leq_P L_2$, which translates instances so that $x \in L_1 \Leftrightarrow f(x) \in L_2$. This makes is possible to compare different problems difficulty and transfer hardness. A problem is NP-complete if it's in NP and every problem in NP reduces to it which hints that a polynomial time algorithm for any NP-complete problem would yield $P = NP$.

II. Preliminaries

In this section, a formal model that is used in NP-completeness theory is shown. Following the the book, computational problems are treated like decision problems where we say yes or no to answer a question, efficiency is measured by polynomial time and complexity classes P and NP are defined in a way that will support hardness proofs by using reductions.

A. Decision Problems and Polynomial Time

There are a lot of problems in the real world that are naturally presented like optimization problems like finding the shortest path or maximizing a profit for a given stock portfolio. Unlike optimization problems, NP-completeness is defined specifically for decision problems where we want to find an answer of whether a solution meeting a specified bound exists or not. For example, instead of asking for the shortest path, we ask:

$$\text{PATH} = \{\langle G, u, v, k\rangle \mid \text{there exists a path from } u \text{ to } v \text{ using at most } k \text{ edges}\}.$$

This way of looking at decisions is important because it fits with a "verifiable certificate" view of computation. A "verifiable certificate" is a proposed solution that can be treated as a witness that the correct answer is "yes."

The book uses polynomial time as the benchmark for efficient computation. Informally, an algorithm is efficient if its worst case running time grows equal or less than n^c for some constant c, where

n is the input size. This polynomial time notion is robust across standard representations and machine models so classifying problems based on if they can be solved in polynomial time is meaningful.

In the language based model, a decision problem corresponds to a language $L \subseteq \{0,1\}^*$ consisting of exactly the "yes" instances. A language L is in P if there exists an algorithm that decides membership in L in polynomial time:

$$\mathbf{P} = \{\, L \subseteq \{0,1\}^* \mid \exists \text{ a polynomial time algorithm that decides } L \,\}.$$

B. Polynomial Time Verification and Class NP

An idea from this chapter is that a lot of difficult problems become easy once a correct solution is provided. CLRS formalize this by using verifiers and certificates. A verifier is a polynomial time algorithm $A(x, y)$ that takes two inputs:

- x: the problem instance
- y: a certificate/witness that should convince the verifier that x is a "yes" instance

The language verified by A is:

$$L = \{\, x \in \{0,1\}^* \mid \exists y \in \{0,1\}^* \text{ s.t. } A(x,y) = 1 \,\}.$$

The important part is that certificates are polynomially bounded in the input length so verification is efficient.

NP is defined as the class of languages with "yes" instances have polynomial size certificates verifiable in polynomial time:

$$\mathbf{NP} = \{L \subseteq \{0,1\}^* \mid \exists \text{ poly time } A,\ \exists c,\ L = \{x : \exists y,\ |y| = O(|x|^c),\ A(x,y) = 1\}\}.$$

In a more intuitive way, NP means "solutions are efficiently checkable." The Hamiltonian cycle mentioned in the introduction is a good example where a proposed cycle is a certificate and verification checks vertex uniqueness and edge existence in polynomial time. Every problem solvable in polynomial time is also verifiable in polynomial time. This means if $L \in \mathbf{P}$ then a verifier can just ignore the certificate and run whatever polynomial decider is available. Therefore, $\mathbf{P} \subseteq \mathbf{NP}$. Whether $P = NP$ is one of the central open questions in theoretical computer science. Chapter 34 uses NP-completeness like a way to understand this question without having to actually solve it directly.

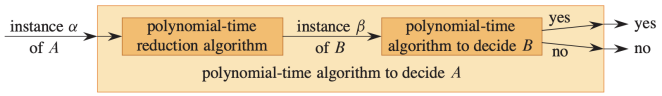


Figure 34.1 How to use a polynomial-time reduction algorithm to solve a decision problem A in polynomial time, given a polynomial-time decision algorithm for another problem B. In polynomial time, transform an instance α of A into an instance β of B, solve B in polynomial time, and use the answer for β as the answer for α.

Fig. 1. P, NP, co-NP relationships.

III. Reductions and NP-Completeness

Rather than proving a problem is hard from scratch, NP-completeness can be used to relate one problem to another using polynomial time reductions. CLRS uses reductions as the main technique for transferring tractability and hardness between decision problems.

A. Polynomial-Time Reductions

To define a many to one reduction let $L_1, L_2 \subseteq \{0,1\}^*$ be languages. We say that L_1 is polynomial time reducible to L_2, $L_1 \leq_P L_2$ and if there exists a function $f : \{0,1\}^* \to \{0,1\}^*$ s.t. f is computable in polynomial time and for all strings x, $x \in L_1 \Leftrightarrow f(x) \in L_2$. The function f works as the reduction since it transforms L_1 into L_2 while preserving the yes or no answer.

The intuition for this is that a reduction formalizes the idea that if L_2 is easy then L_1 is easy. If we can compute $f(x)$ quickly and then decide whether $f(x) \in L_2$ then we get an algorithm for deciding whether $x \in L_1$. CLRS states this as a lemma where if $L_1 \leq_P L_2$ and $L_2 \in \mathbf{P}$ then $L_1 \in \mathbf{P}$. The proof is just "compute $f(x)$ then run the decider for L_2."

With a reduction, a valid reduction proof has to be proven with two parts. First, correctness (iff) where you proof $x \in L_1 \Leftrightarrow f(x) \in L_2$. This is can be shown either if $x \in L_1$ then $f(x) \in L_2$ or if $f(x) \in L_2$ then $x \in L_1$. Second, efficiency where you have to proof f runs in time polynomial in $|x|$ (and usually that $|f(x)|$ is polynomially bounded in $|x|$). The reductions matter because they preserve hardness. To show a problem B is hard, we reduce a known hard problem A to B so hardness flows

from A to B. If $A \leq_P B$ then any fast algorithm for B would imply a fast algorithm for A.

B. NP-Hardness and NP-Completeness

Reductions are more useful when we identify a class of problems that are "as hard as" the hardest problems in NP. A language L is NP-hard if every language $L' \in \mathbf{NP}$ reduces to L where $\forall L' \in \mathbf{NP},\ L' \leq_P L$. A language L is NP-complete if it's NP-hard and also belongs to NP where $L \in \mathbf{NP}$ and $\forall L' \in \mathbf{NP},\ L' \leq_P L$.

NP-complete problems are important because they are in NP but also NP hard. Meaning, the problem has solutions that are efficiently verifiable but they are at least as hard as any problem with efficiently verifiable solutions. The implication is that if any NP-complete problem is solvable in polynomial time then $P = NP$. Also, if $P \neq NP$ then no NP-complete problem has a polynomial time algorithm. This is why NP-completeness is used to explain that efficient algorithms are not likely in general.

Furthermore, NP-complete problem can enable many others. NP-hardness looks impossible to prove directly because it quantifies over all languages in NP. CLRS resolves this by checking (Lemma 34.8) if A is NP-complete and $A \leq_P B$ then B is NP-hard. Also if $B \in \mathbf{NP}$ then B is NP-complete. It makes sense since every $L' \in \mathbf{NP}$ reduces to A and A reduces to B then every L' reduces to B.

As a result, NP-completeness something we can use again and again. Once we have a small set of canonical NP-complete problems, new NP-completeness proofs only really need to show membership in NP and giving a single polynomial time reduction from a known NP-complete problem.

C. Polynomial-Time Reductions

IV. NP-Complete Problems

NP-completeness proofs depend on problems that are already established as NP-complete. CLRS shows a pipeline that starts with Boolean circuits as a computational model then to Boolean formulas and then standardizes the input structure into a normal form.

A. CIRCUIT-SAT, SAT and 3-CNF-SAT

For problem CIRCUIT-SAT, a Boolean combinational circuit C takes Boolean inputs and outputs a single Boolean value. This problem asks whether there exists an input assignment that makes the circuit output 1. CIRCUIT-SAT is a good first NP-complete problem because circuits can encode polynomial time computation and verification. More specifically, given a verifier that runs in polynomial time, its computation can be represented in polynomial size as a circuit with satisfiability matches the existence of an accepting certificate.

For problem SAT, a Boolean formula φ is satisfiable if there exists a truth assignment to its variables that makes φ be true. This problem asks whether a given formula is satisfiable. SAT is NP-complete and easy to work with because formulas are a familiar mathematical object. The key part in CLRS is that converting a circuit directly into a formula can cause exponential blowup because of fan out. The basic fix is to use variables for wires and constraints for gates that makes a polynomial size formula equivalent to the circuit.

For problem 3-CNF-SAT (3SAT), a formula is in conjunctive normal form (CNF) if it's an AND of clauses and each clause is an OR of literals. A formula is in 3-CNF if every clause has exactly three literals. This problem asks whether such a formula is satisfiable. CLRS shows 3SAT is NP-complete by reducing SAT to 3SAT while preserving satisfiability and keeping the instance size polynomial.

Moreover, SAT and 3SAT are standard reduction starting points. SAT and 3SAT serve like convenient reductions because they encode constraint satisfaction in a uniform, modular way. They show a complex problem can be seen as choosing objects subject to constraints and Boolean formulas express constraints locally (clause by clause). Also a lot of reductions go by building a target instance with a structure that forces a solution to match choosing one satisfying literal per clause, one configuration per variable, etc. This uniformity makes 3SAT a good source problem for reductions for things like graphs, sets, paths and optimization constraints.

B. NP-Completeness Proof: $3SAT \leq_P CLIQUE$

Here, we show a complete NP-complete proof with the goal to show the methodology. The problem

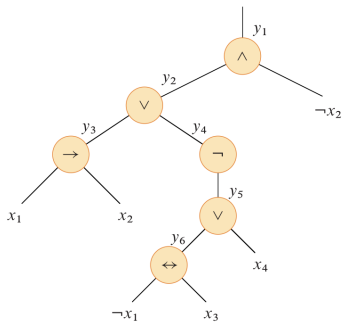


Figure 34.11 The tree corresponding to the formula $\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$.

Fig. 2. 3-CNF-SAT to CLIQUE reduction.

we want is CLIQUE. A clique in an undirected graph $G = (V, E)$ is a subset $S \subseteq V$ s.t. every pair of distinct vertices in S is connected by an edge. The decision problem is

$$\text{CLIQUE} = \{\langle G, k\rangle \mid G \text{ contains a clique of size } k\}.$$

We claim that CLIQUE is NP-complete.

It's enough to show that CLIQUE $\in$ **NP** and 3SAT $\leq_P$ CLIQUE to proof this claim. With a certificate of a set $S \subseteq V$ of vertices and a verifier that given $\langle G, k\rangle$ and certificate S, it can verify $|S| = k$ and for every distinct $u, v \in S$ check that $(u, v) \in E$. There are $\binom{k}{2}$ pairs so verification runs in time polynomial in the input size. Therefore, CLIQUE $\in$ **NP**.

To make a reduction from 3SAT to CLIQUE, let the input be a 3-CNF formula with k clauses as $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$ where each clause has exactly three literals $C_i = (\ell_{i1} \vee \ell_{i2} \vee \ell_{i3})$. We will construct an undirected graph G s.t. φ is satisfiable $\Leftrightarrow \langle G, k\rangle \in$ CLIQUE. To construct a graph $G = (V, E)$, we need to focus on the vertices and the edges. For the vertices, we can create one vertex for each literal occurrence in each clause as

$$V = \{v_{ij} \mid 1 \leq i \leq k,\ 1 \leq j \leq 3\}$$

so $|V| = 3k$. Here, we view v_{ij} like picking literal ℓ_{ij} as the satisfying literal for clause C_i. For the edges, we can connect vertices from different clauses when their literals are consistent. Specifically, for $i \neq r$ add an edge between v_{ij} and v_{rs} iff ℓ_{ij} is not the negation of ℓ_{rs} (ie. $(v_{ij}, v_{rs}) \in E \Leftrightarrow i \neq r$ and $\ell_{ij} \neq \neg\ell_{rs}$). For the output, we just return the CLIQUE instance $\langle G, k\rangle$.

This algorithm is polynomial time bound since the graph has $3k$ vertices. It just checks pairs of vertices from different clauses and adds edges based on a simple literal complement test. The reduction is polynomial in the formula size so the construction runs in polynomial time.

To proof for correctness, we prove that φ satisfiable $\Leftrightarrow G$ has a clique of size k. To show if φ is satisfiable then G has a k clique, we assume φ is satisfiable under some truth assignment. Then every clause C_i has at least one literal that evaluates to true. For each clause C_i, choose one true literal ℓ_{ij} and select the corresponding vertex v_{ij}. Let

$$S = \{v_{1j_1}, v_{2j_2}, \ldots, v_{kj_k}\}.$$

Then $|S| = k$ and S contains one vertex from each clause. Now take any two distinct vertices v_{ij_i} and v_{rj_r} in S. Since both chosen literals are true under the same assignment, they can't be complements of each other (it's impossible for both x and $\neg x$ to be true at the same time). Based on the construction this means that $(v_{ij_i}, v_{rj_r}) \in E$. Therefore every pair in S is connected so S is a clique of size k.

To show if G has a k clique then φ is satisfiable, we assume G has a clique S with $|S| = k$. Since edges only connect vertices from different clauses, S can't contain two vertices from the same clause. Since there are exactly k clauses total the clique must contain exactly one vertex from each clause as $S = \{v_{1j_1}, v_{2j_2}, \ldots, v_{kj_k}\}$. Now assign truth values so each selected literal is true. This assignment is consistent because S is a clique where for any two selected literals ℓ_{ij_i} and ℓ_{rj_r} the edge between v_{ij_i} and v_{rj_r} exists (it means the literals are not complements). Hence we never force both x and $\neg x$ to be true. Under this assignment, every clause C_i contains its selected literal ℓ_{ij_i} which is true so every clause is satisfied. Therefore φ is satisfiable.

We have shown that a CLIQUE $\in$ **NP** by having a polynomial time verifier for a size k clique certificate. We have also shown 3SAT $\leq_P$ CLIQUE by using the construction above with a polynomial runtime and a full iff correctness argument. Since 3SAT is NP-complete, we can also say that CLIQUE is NP-hard and NP-complete (since CLIQUE is in NP).

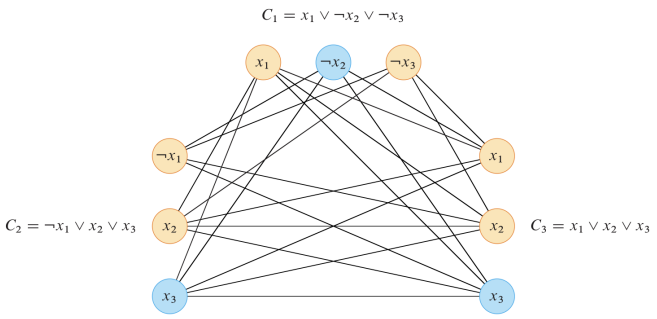


Figure 34.14 The graph G derived from the 3-CNF formula $\phi = C_1 \wedge C_2 \wedge C_3$, where $C_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$, $C_2 = (\neg x_1 \vee x_2 \vee x_3)$, and $C_3 = (x_1 \vee x_2 \vee x_3)$, in reducing 3-CNF-SAT to CLIQUE. A satisfying assignment of the formula has $x_2 = 0$, $x_3 = 1$, and x_1 set to either 0 or 1. This assignment satisfies C_1 with $\neg x_2$, and it satisfies C_2 and C_3 with x_3, corresponding to the clique with blue vertices.

Fig. 3. NP-complete reduction chain.

V. Implications for Algorithm Design

NP-completeness is not just useful for theoretical classification. It has a direct impact on what algorithm designers should try to achieve (or stay away from). After a problem is presented as NP-complete then it's NP-hard and every problem in NP reduces to it. Because of this, an exact, polynomial time algorithm for a problem like that would mean $P = NP$. There are a lot of debates over the equality being true or false but NP-completeness is seen as there isn't a polynomial time algorithm for a problem in general.

However, it doesn't mean that NP-complete problems don't matter in the real world. How we do algorithm design become more relevant. Instead of wasting time looking for exact solutions, algorithm designers can find other goals that better show computational limits. For optimization problems, this looks more like approximation algorithms that run in polynomial time and give solutions close to optimal. Heuristic methods are also good in using algorithms like greedy search or randomized algorithms. We are able to do these things because NP-completeness suggests that worst case efficiency probably won't happen.

Another argument is that we can restrict the problem domain. A lot of NP-complete problems become tractable when inputs have better or a modified structure or maybe something becomes bounded. Figuring out what these are and where they fit best makes it possible to develop exact polynomial time algorithms for subclasses that can be useful even if the general problem stays hard. Because of this, NP-completeness cane be seen kind of like a guide where it clarifies the natural difficulty of a problem and helps designers choose between approximations, heuristics or restricted formulas than just going straight for a general solution.

VI. Conclusion

This report showed an overview of the NP-completeness framework that is covered in Chapter 34 of Introduction to Algorithms. From this chapter, we mainly gather that computational problems can be classified by tractability using polynomial time to mark efficiency. Because of this, we can group together problems based on a defined difficulty even if they problems themselves don't look similar.

We formalized problems as decision problems and showed what P and NP are. The verifier/certificate definition of NP shows why a lot of problems that lack a polynomial time solution still have a fast verification. Reductions show how we can compare how difficult a given problem is which gives the definitions of NP-hardness and NP-completeness. A problem A problem is NP-complete when it's NP and at least as hard as every other problem in NP.

All of this is shown with the NP-complete proof for the CLIQUE problem using a 3SAT reduction. Doing this lets us see how the standard structure of NP-complete proofs work. Finally, the report covered how NP-completeness affects algorithms and algorithm designers.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.